

Servlets

Unit-3

Topics:

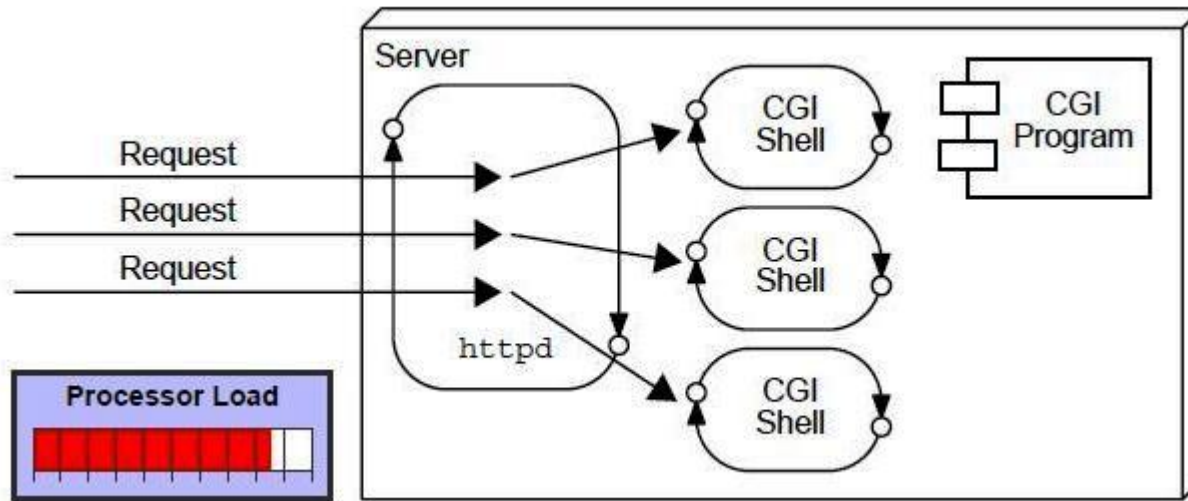
- 3.1 Fundamentals of Java Servlet programming
- 3.2 A simple java Servlet
- 3.3 Servlet life cycle
- 3.4 Developing and Deploying Servlets
- 3.5 Working with cookies

BASIC SERVLET STRUCTURE:

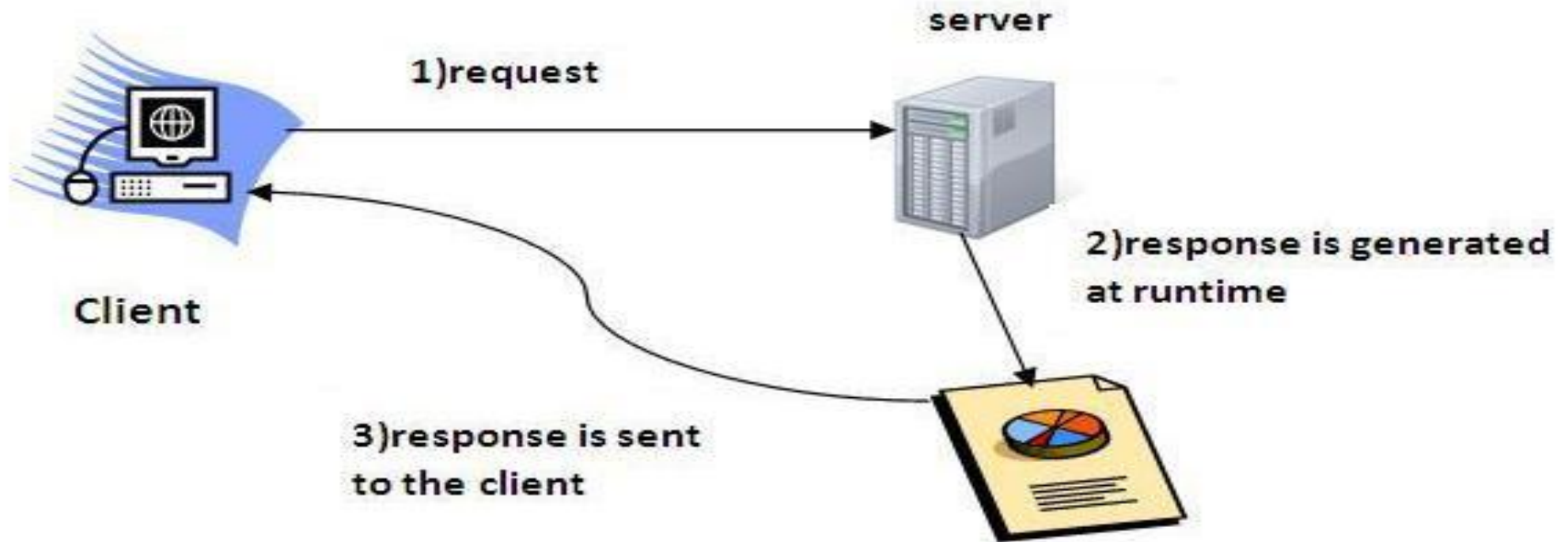
- **Servlet** technology is used to create web application (resides at server side and generates dynamic web page).
- There are many interfaces and classes in the servlet API such as **Servlet**, **GenericServlet**, **HttpServlet**, **ServletRequest**, **ServletResponse** etc

HOW SERVLET DIFFER FROM CGI:

- **CGI(Common Gateway Interface)**
- CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.

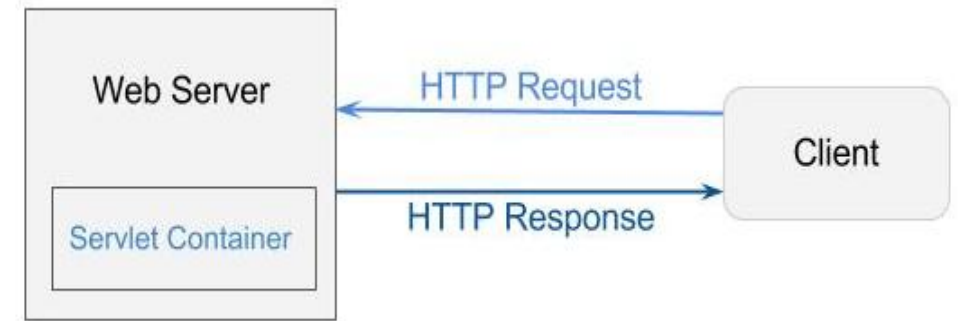


Servlet:



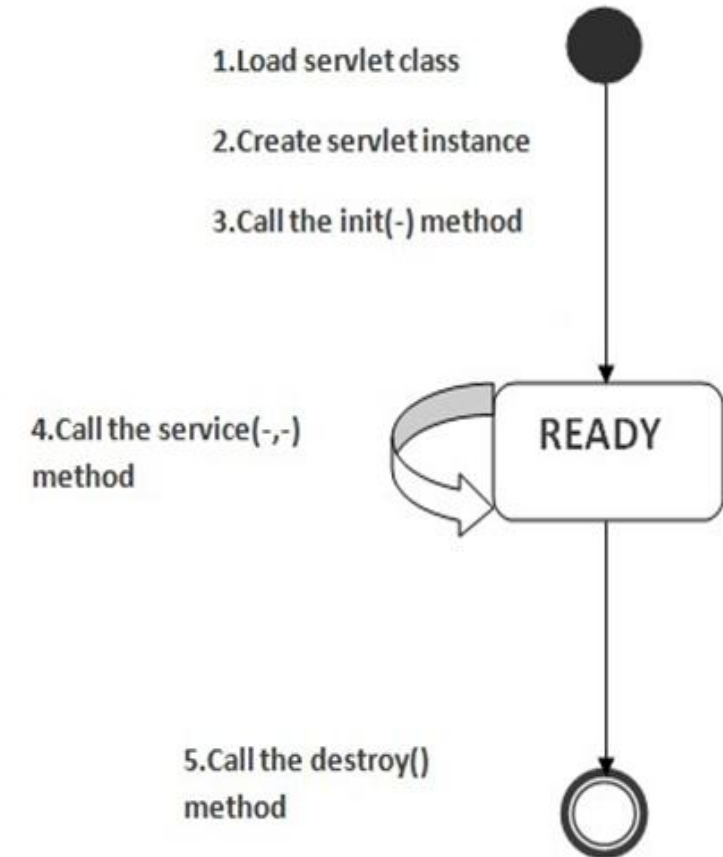
Servlet Container:

- It provides the runtime environment for JavaEE (j2ee) applications. **The client/user can request only a static WebPages from the server.**
- If the user wants to read the web pages as per input then the servlet container is used in java.
- The servlet container is used in **java for dynamically generate the web pages on the server side.**
- Therefore the servlet container is the part of a **web server that interacts with the servlet for handling the dynamic web pages from the client.**



LIFE CYCLE OF SERVLET:

- The web container maintains the life cycle of a servlet instance.
1. Servlet class is loaded.
 2. Servlet instance is created.
 3. init method is invoked.
 4. service method is invoked.
 5. destroy method is invoked.



LIFE CYCLE OF SERVLET:

- **1) Servlet class is loaded**

- The classloader is responsible to load the servlet class.
- The servlet class is loaded when the first request for the servlet is received by the web container.

- **2) Servlet instance is created**

- The web container creates the instance of a servlet after loading the servlet class.
- The servlet instance is created only once in the servlet life cycle.

3) init method is invoked

- The web container calls the init method only once after creating the servlet instance.
- The init method is used to initialize the servlet.
- It is the life cycle method of the javax.servlet.Servlet interface.

Syntax of the init method is given below:

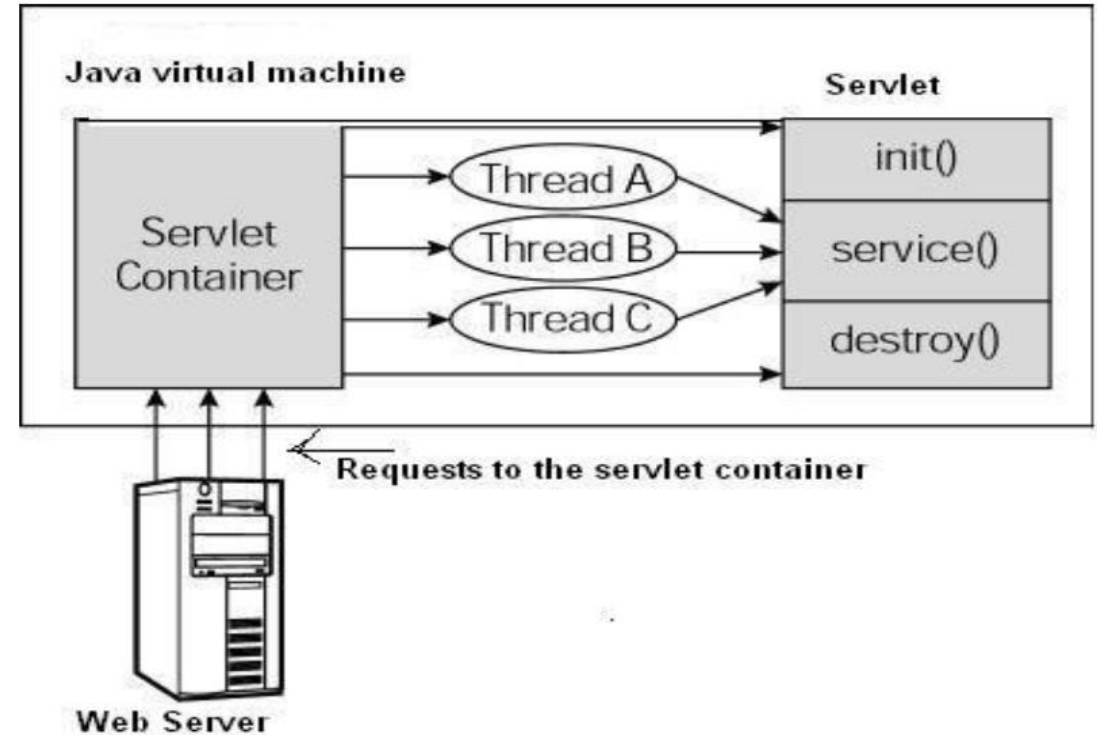
- 1. **public void init(ServletConfig config) throws ServletException**

4) service method is invoked

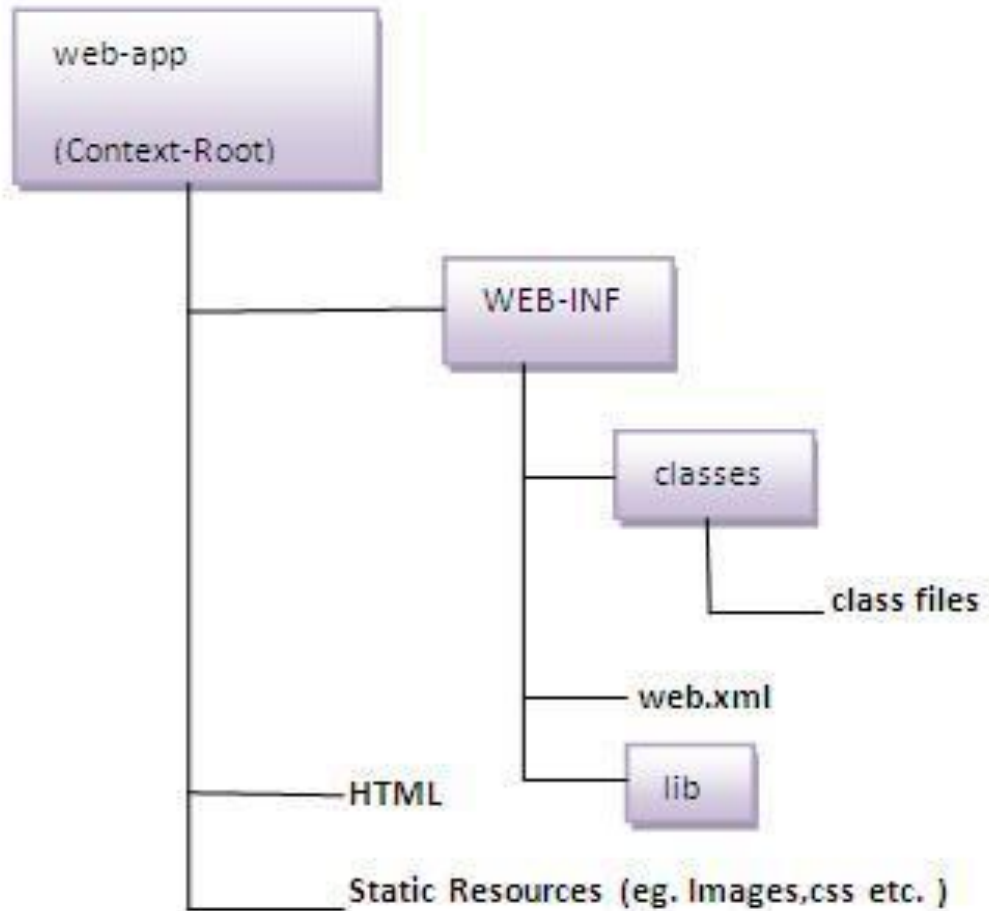
- The web container calls the service method each time when request for the servlet is received.
- If servlet is not initialized, it follows the first three steps as described above then calls the service method.
- If servlet is initialized, it calls the service method.
- The syntax of the service method
:public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException

5) destroy method is invoked.

- The web container calls the destroy method before removing the servlet instance from the service.
- It gives the servlet an opportunity to clean up any resource for example memory, thread etc.
- The syntax of the destroy method of the Servlet interface is given below:
 - `public void destroy()`



EXECUTION PROCESS OF SERVLET APPLICATION:



1. Create a directory structure
2. Create a Servlet
3. Compile the Servlet
4. Create a deployment descriptor
5. Start the server and deploy the project
6. Access the servlet

2) Create a Servlet:

There are three ways to create the servlet.

- **By implementing the Servlet interface**
- **By inheriting the GenericServlet class**
- **By inheriting the HttpServlet class**
- The HttpServlet class is widely used to create the servlet because it provides methods to **handle http requests such as doGet(), doPost, doHead() etc**

```
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;

public class DemoServlet extends HttpServlet{
    public void doGet(HttpServletRequest req,HttpServlet
    Response res)
    throws ServletException,IOException
    {
        res.setContentType("text/html");//setting the conten
        t type
        PrintWriter pw=res.getWriter();//get the stream to wr
        ite the data

        //writing html in the stream
        pw.println("<html><body>");
        pw.println("Welcome to servlet");
        pw.println("</body></html>");

        pw.close();//closing the stream
    }
}
```

3)Compile the servlet:

For compiling the Servlet, jar file is required to be loaded.

Different Servers provide different jar files:

Jar file	Server
1) servlet-api.jar	Apache Tomcat
2) weblogic.jar	Weblogic
3) javaee.jar	Glassfish
4) javaee.jar	JBoss

Two ways to load the jar file

1. **set classpath**
2. **paste the jar file in JRE/lib/ext folder**

Put the java file in any folder. After compiling the java file, paste the class file of servlet in WEB-INF/classes directory.

- **4)Create the deployment descriptor (web.xml file):**The **deployment descriptor** is an xml file, from which Web Container gets the information about the servlet to be invoked.

web.xml file

<web-app>

<servlet>

<servlet-name>s1</servlet-name>

<servlet-class>DemoServlet</servlet-class>

</servlet>

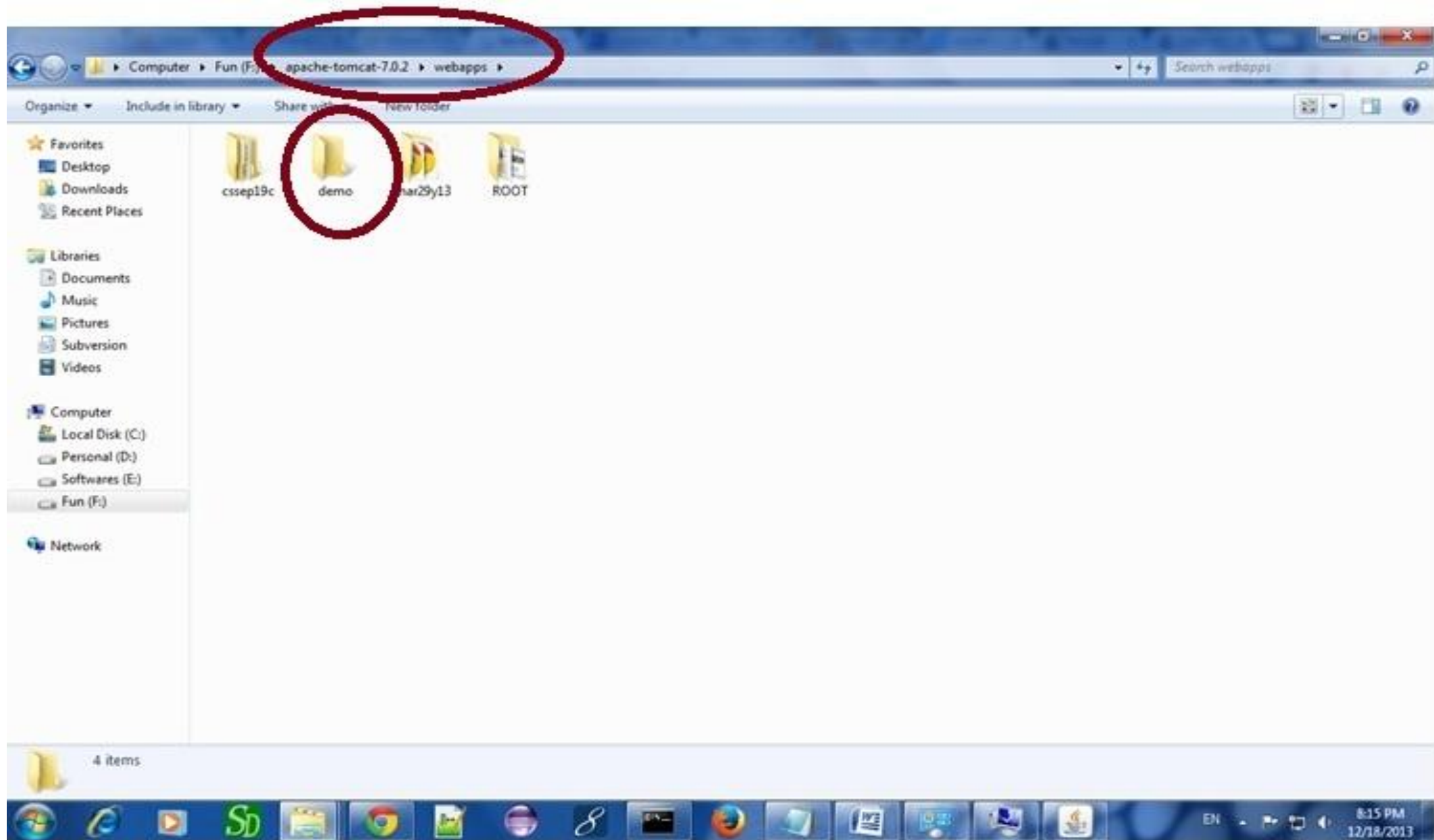
<servlet-mapping>

<servlet-name>s1</servlet-name>

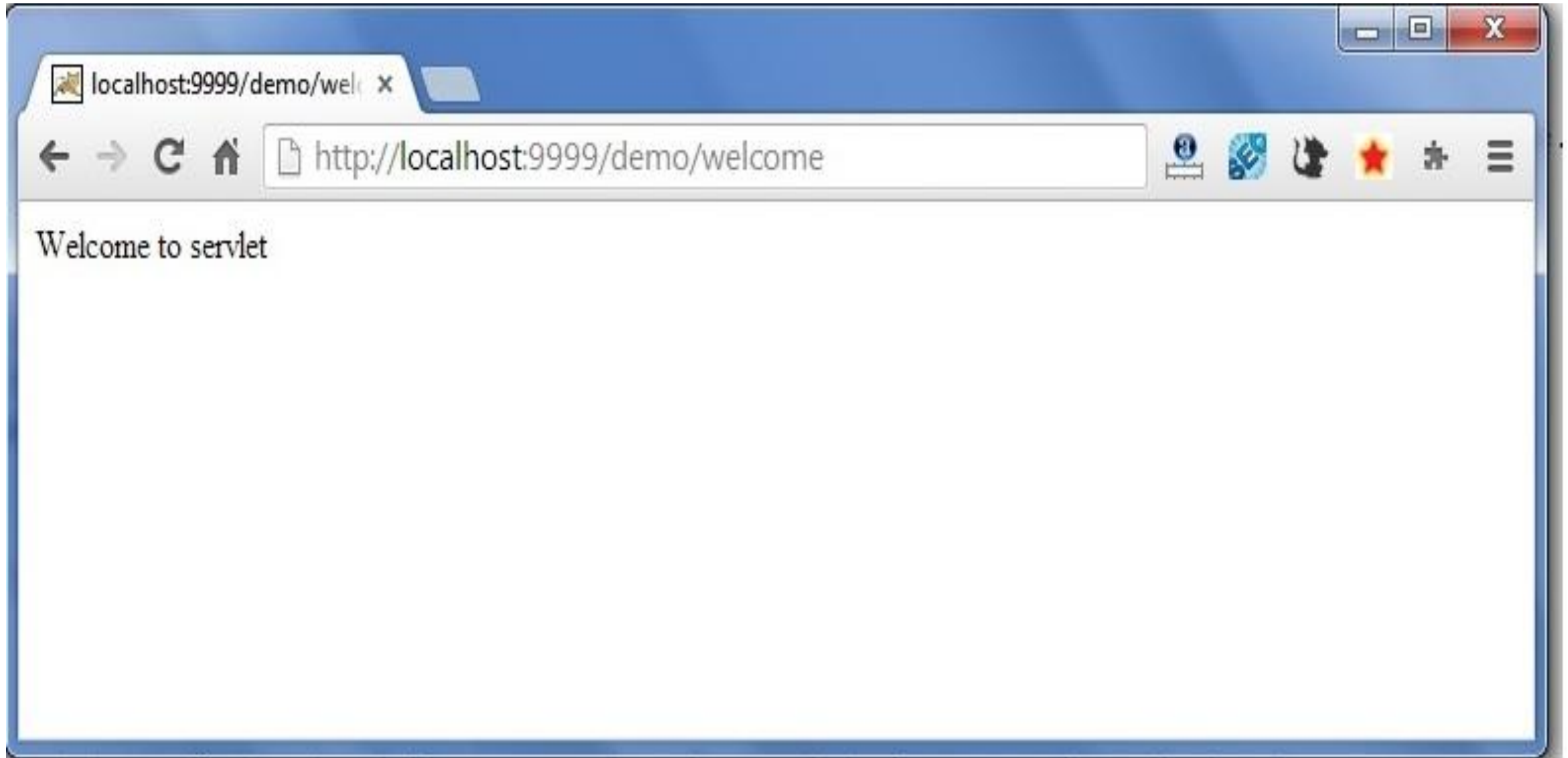
<url-pattern>/welcome</url-pattern>

</servlet-mapping>

</web-app>



How to access the servlet:<http://localhost:9999/demo/welcome>



TYPES OF SERVLET:

1. **Servlet interface** provides common behavior to all the servlets.
 - Servlet interface needs to be implemented for creating any servlet (either directly or indirectly).
 - It provides 3 life cycle methods that are used to initialize the servlet, **to service the requests, and to destroy the servlet and 2 non-life cycle methods.**

Methods of Servlet interface:

- There are 5 methods in Servlet interface. The init, service and destroy are the life cycle methods of servlet. These are invoked by the web container.

Method	Description
<code>public void init(ServletConfig config)</code>	initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.
<code>public void service(ServletRequest request, ServletResponse response)</code>	provides response for the incoming request. It is invoked at each request by the web container.
<code>public void destroy()</code>	is invoked only once and indicates that servlet is being destroyed.
<code>public ServletConfig getServletConfig()</code>	returns the object of ServletConfig.
<code>public String getServletInfo()</code>	returns information about servlet such as writer, copyright, version etc.

2. GENERIC SERVLET:

- **GenericServlet** class implements **Servlet**, **ServletConfig** and **Serializable** interfaces.
- It provides the implementation of all the methods of these interfaces except the service method.
- GenericServlet class can handle any type of request so it is protocol-independent.
- You may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.

Methods of GenericServlet class:

- There are many methods in GenericServlet class. They are as follows:
- **public void init(ServletConfig config)** is used to initialize the servlet.
- **public abstract void service(ServletRequest request, ServletResponse response)** provides service for the incoming request. It is invoked at each time when user requests for a servlet.
- **public void destroy()** is invoked only once throughout the life cycle and indicates that servlet is being destroyed.
- **public ServletConfig getServletConfig()** returns the object of ServletConfig.
- **public String getServletInfo()** returns information about servlet such as writer, copyright, version etc.

3. HTTP SERVLET:

- The HttpServlet class extends the GenericServlet class and implements Serializable interface.
- It provides http specific methods such as doGet, doPost, doHead, doTrace etc.

Methods of HttpServlet class:

There are many methods in HttpServlet class. They are as follows:

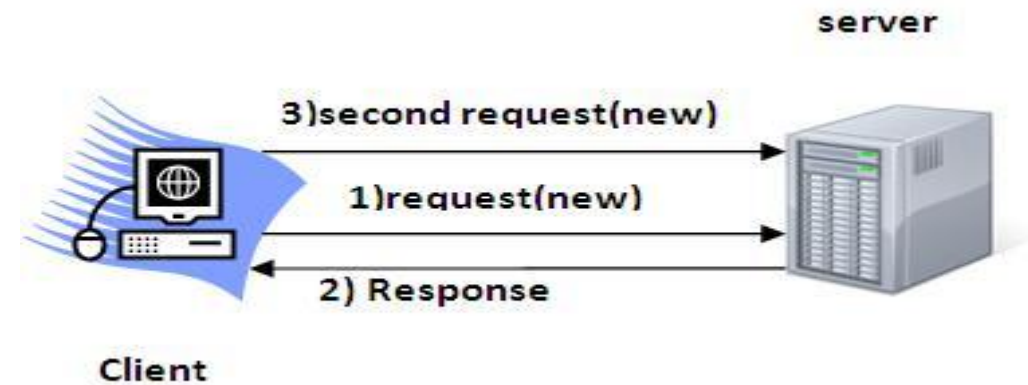
1. **public void service(ServletRequest req,ServletResponse res)** dispatches the request to the protected service method by converting the request and response object into http type.
2. **protected void service(HttpServletRequest req, HttpServletResponse res)** receives the request from the service method, and dispatches the request to the doXXX() method depending on the incoming http request type.
3. **protected void doGet(HttpServletRequest req, HttpServletResponse res)** handles the GET request. It is invoked by the web container.
4. **protected void doPost(HttpServletRequest req, HttpServletResponse res)** handles the POST request. It is invoked by the web container.

SESSION TRACKING:

Session Tracking in Servlets

- [Session Tracking](#)
- [Session Tracking Techniques](#)
- **Session** simply means a particular interval of time.
- **Session Tracking** is a way to maintain state (data) of an user. It is also known as **session management** in servlet.

- Http protocol is a stateless so we need to maintain state using session tracking techniques.
- Each time user requests to the server, server treats the request as the new request.
- So we need to maintain the state of an user to recognize to particular user.
- HTTP is stateless that means each request is considered as the new request.



Why use Session Tracking?

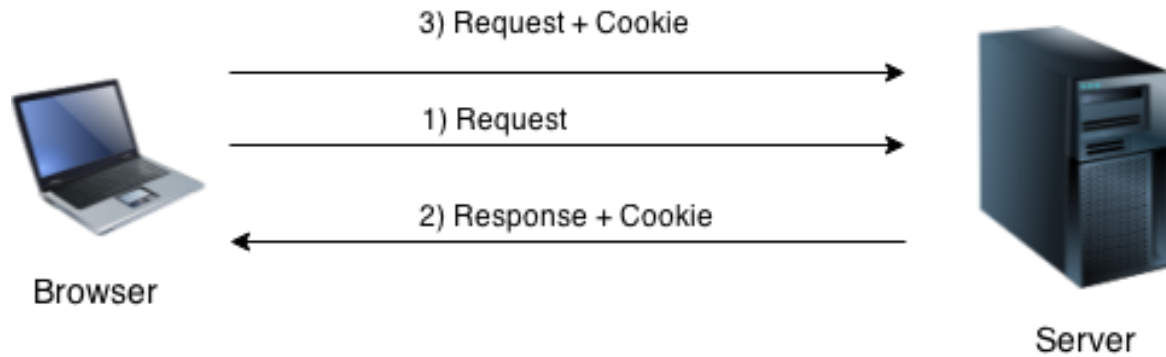
- **To recognize the user** It is used to recognize the particular user.
- **Session Tracking Techniques**
 - **Cookies:**A webserver can assign a unique session ID as a cookie to each web client and for subsequent requests from the client they can be recognized using the recieved cookie.

COKKIE CLASS

- Cookies are text files stored on the client computer and they are kept for various information tracking purpose.

- There are three steps involved in identifying returning users:
 1. Server script sends a set of cookies to the browser. **For example name, age, or identification number etc.**
 2. Browser **stores this information on local machine for future use.**
 3. When next time browser sends any request to web server then it **sends those cookies information to the server and server uses that information to identify the user.**

How Cookie works



• Types of Cookie

There are 2 types of cookies in servlets.

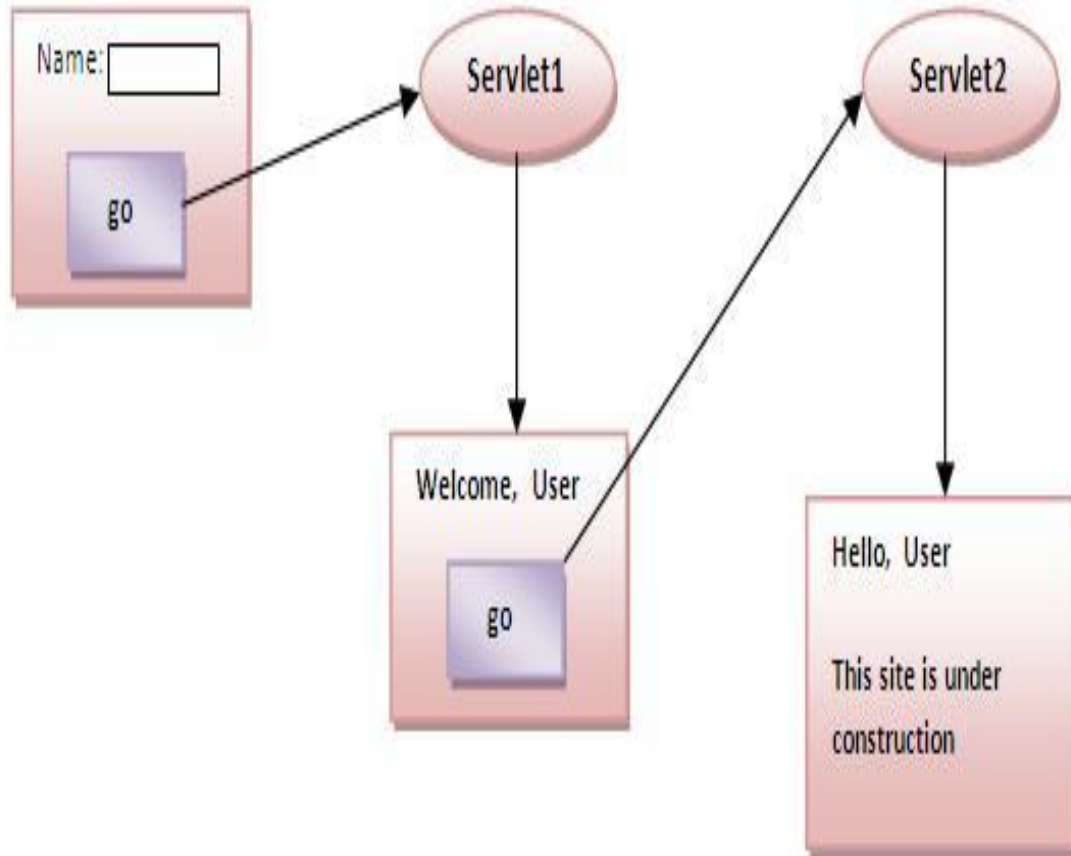
1. Non-persistent cookie
2. Persistent cookie

1. Non-persistent cookie

It is **valid for single session** only. It is removed each time when user closes the browser.

2. Persistent cookie

It is **valid for multiple session** . It is not removed each time when user closes the browser. It is removed only if user logout or signout.



index.html

1. `<form action="servlet1" method="post">`
2. `Name:<input type="text" name="userName"/><br/>`
3. `<input type="submit" value="go"/>`
4. `</form>`